6. Bölüm
Kontrol İşlemleri

6.1 Ardışık İşlem ve Kontrol İşlemleri

Yapısal Programlama başlığı altında programın ana fonksiyondan başlayacağı ve programlamanın ise birbirini çağıran fonksiyonlarla yapıldığı anlatılmıştı. Yapısal programlamada, ana fonksiyon dahil tüm fonksiyonlarda ilk önce işlenecek verileri taşıyan veri yapıları tanımlanır ve ardından bu verileri işleyen kontrol yapılarına ilişkin talimatlar yazılır.

Şu ana karar örneğini verdiğimiz kodlarda veri yapısı olarak sadece değişkenler kullanılmıştır. Sonrasında ise giriş çıkış işlemleri talimatlar içeren programlar yazılmıştır. Programın icrası ilk talimattan başlar ve sırasıyla program bitene kadar devam eder. İşte programın icrasını değiştirmeyen bu tür talimatlara **ardışık işlem** (sequential operation) adı verilir.

Ardışık işlemler aşağıdaki üç tür talimattan (statement) oluşur.

1. Değişkenlerin kimliklendirildiği değişken tanımlamaları (identifier definition):
   (a) `int yariCap=3;`
   (b) `const float PI=3.14;`
   (c) `float daireninAlani,daireninCevresi;`
2. **İfadelerden** (expression) yani sabit, değişken ve operatör içeren söz dizimleri:
   (a) `daireninAlani=PI*yariCap*yariCap;`
   (b) `float daireninCevresi=2.0*PI*yariCap;`
3. Klavyeden veri okuma ve konsola veri yazma gibi giriş-çıkış işlemleri (input-output operation):
   (a) `printf("Kapasite Oranını (0.00-1.00) Giriniz:");`
   (b) `scanf("%f",&kapasiteOrani);`

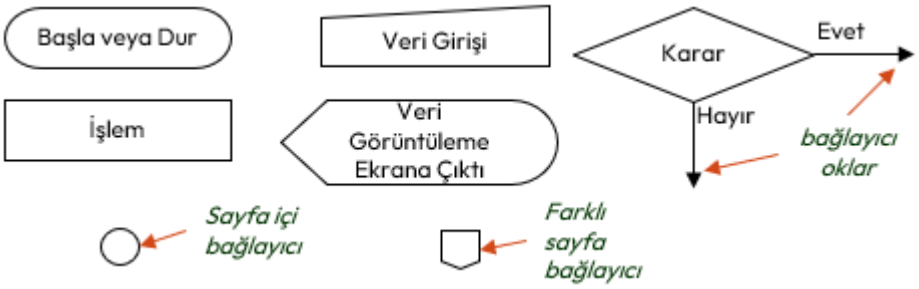
Kontrol işlemleri (control operation) ise programın icra sırasını değiştiren kontrol yapıları olup bu talimatlar (statement) olup üç türü vardır;

1. **Duruma göre seçimler** (conditional choice): `if`, `if-else` ve `switch` talimatları.
2. **İlişkisel döngü** (relational loop): `while`, `do-while` ve `for` talimatları.
3. **Dallanmalar** (jump): `continue`, `break`, `goto` ve `return` talimatları.

Kontrol işlemlerinde; **kodlanan mantıksal satırlar (logical sequence) ile fiziksel olarak icra edilen satırlar (physical sequence) birbirinden farklıdır.**

6.2 Akış Diyagramları

Kontrol işlemi içeren programın icrasını anlamak için akış diyagramlarını da anlamak gerekir. Aşağıda akış diyagramlarındaki temel şekiller verilmiştir.

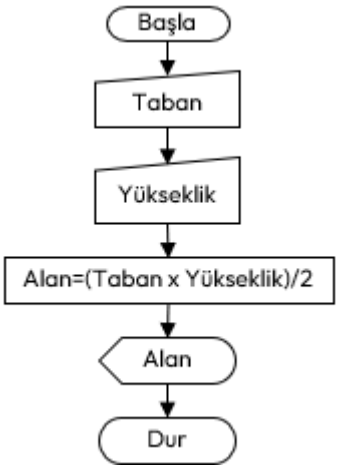


Şekil 6.1: Akış Diyagramları Genel Şekilleri

Akış diyagramları (flowchart), adımların grafik gösterimleridir. Algoritmaları ve programlama mantığını temsil etmek için bir araç olarak bilgisayar biliminden ortaya çıkmıştır. Ancak diğer tüm işlem türlerinde kullanılmak üzere genişletilmiştir. Günümüzde akış diyagramları, bilgileri göstermede ve akıl yürütmeye yardımcı olmada son derece önemli bir rol oynamaktadır. Karmaşık süreçleri görselleştirmemize veya sorunların ve görevlerin yapısını açık hale getirmemize yardımcı olurlar. Bir akış şeması, uygulanacak bir süreci

veya projeyi tanımlamak için de kullanılabilir.

Aşağıda taban ve yüksekliği klavyeden girilen bir dik üçgenin alanını hesaplamak için bir akış diyagramı örneği verilmiştir.



Şekil 6.2: Dik Üçgenin Alana Hesabını Gösteren Akış Diyagramı

Akış diyagramı çizilirken aşağıdaki kurallara uyulur;

- Akış şemalarında tek bir başlangıç simgesi olmalıdır
- Bitiş simgesi birden çok olabilir.
- Karar simgesinin haricindeki simgelere her zaman tek giriş ve tek çıkış yolu bulunur.
- Bağlaç simgesi sayfanın dolmasından ötürü parçalanan akış şemasının öğelerini birleştirmede kullanılır.
- Simgeler birbirleri ile tek yönlü okla bağlanırlar.
- Okların yönü algoritmanın mantıksal işlem akışını tanımlar.

6.3 Duruma Göre Seçimler

Duruma göre seçimler (conditional choice) programın akışını bir koşula göre değiştiren talimatlardır.

§6.3.1 If Talimatı

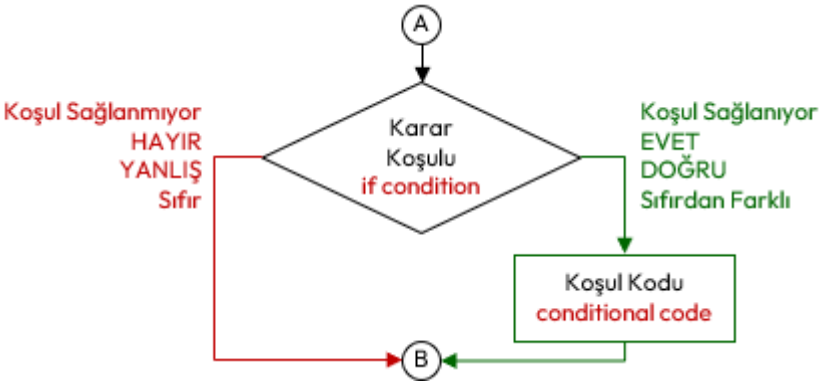
If talimatı, karar vermeye (decision making) ilişkin bir talimat olup bir koşula bağlı olarak programın icrasını değiştirir. If talimatının sözde kodu aşağıda verilmiştir;

```
if (karar-koşulu)
    koşul-kodu;
```

Burada ilişkisel işleçler içeren **karar koşulu** (condition) ifadesi (expression) test edilir. Sonuç DOĞRU ise **koşul kodu** (conditional code) icra edilir, değilse icra edilmez.

> Karar koşulu test sonucu **sıfırdan farklı ise DOĞRU, aksi durumda YANLIŞ kabul edilir**. Bu husus 4.7.3 başlığında anlatılmıştı.

Bu talimatın akış diyagramı da aşağıdaki gibidir;



Şekil 6.3: If Talimatı İcra Akışı

Aşağıdaki örnek programı inceleyelim;

```
1  #include <stdio.h>
2  int main() {
3          int yas=18;
4          char cinsiyet='K';
5          if (yas<30)
6              printf("Genç");
7          if (cinsiyet=='E')
8              printf("Erkek");
9          return 0;
10 }
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

1. Birinci Satırda `printf` fonksiyonu için `stdio.h` başlığı koda dahil edilmiştir.
2. İkinci satırda programın başlayacağı ana fonksiyon tanımlanmıştır.
3. Üçüncü ve dördüncü satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
4. Beşinci satırda bir `if` talimatı bulunmaktadır. **Karar koşulu** (condition), `yas<30` ifadesiyle test edilecektir. Test sonucu 1 yani DOĞRU olduğundan altıncı satırdaki koşul kodu (conditional code) talimatı **icra edilir.** Yani `printf("Genç");` talimatı ile konsola `"Genç"` yazılır.
5. Yedinci satırda da bir `if` talimatı bulunmaktadır. Karar koşulu, `cinsiyet=='E'` ifadesiyle test edilecektir. Test sonucu 0 yani YANLIŞ olduğundan sekizinci satırdaki koşul kodu talimatı **icra edilmez.**
6. Son olarak dokuzuncu satırdaki `return 0;` ifadesi icra edilerek işletim sistemine dönülür.

If talimatında koşul kodu (conditional code) her zaman ardışık işlem (sequential operation) olmaz. Bazen `if` gibi bir başka kontrol işlemi (control operation) de olabilir.

Aşağıda kademeli (compound/cascade) `if` kullanımına ilişkin kod örneğinde ikinci `if`, birinci `if` talimatının koşul kodudur;

```
if (yas<30)
    if (yas<7) // Bu if, üsteki if talimatının koşul kodudur.
        printf("Çoçuk");
```

Koşul kodu (conditional code) birden fazla talimattan oluşacak ise kod bloğu içine alınır. Aşağıda verilen kod örneğinde ilk `if` talimatında `yas<30` ile test edilen koşul doğrulandığında kod bloğunun içindeki tüm talimatlar icra edilir.

```
if (yas<30) { //koşul-kodu bloğu başlangıcı
        if (yas<7)
            printf("Çocuk");
        if (yas<18)
            printf("Genç");
        if (yas>60)
           printf("Yaşlı");
} //koşul-kodu bloğu bitişi
```

Bazı durumlarda bir if, başka bir if bloğu içinde yani iç içe (nested if) olabilir. Buna ilişkin kod örneği de aşağıda verilmiştir.

```
if (cinsiyet=='E') { // dış if bloğu başlangıcı
        if (yas<7) { // iç if bloğu başlangıcı
            printf("Erkek Çocuk ");
            printf("Yada Erkek Bebek");
        } // iç if bloğu başlangıcı
        if (yas<18)
                printf("Genç Delikanlı");
        if (yas>60)
                printf("Yaşlı Adam");
} // dış if bloğu bitişi
```

If talimatında koşul (condition) her zaman tek bir test ifadesinden (expression) oluşmayabilir. Bahsedilen duruma ilişkin kod örneği de aşağıda verilmiştir.

```
if ((cinsiyet=='E') && (yas<18))  // hem Erkek, hem de genç
        printf("Genç Delikanlı");
if ((cinsiyet=='K') && (yas>60)) // hem yaşlı, hem de kadın
        printf("Yaşlı Kadın");
```

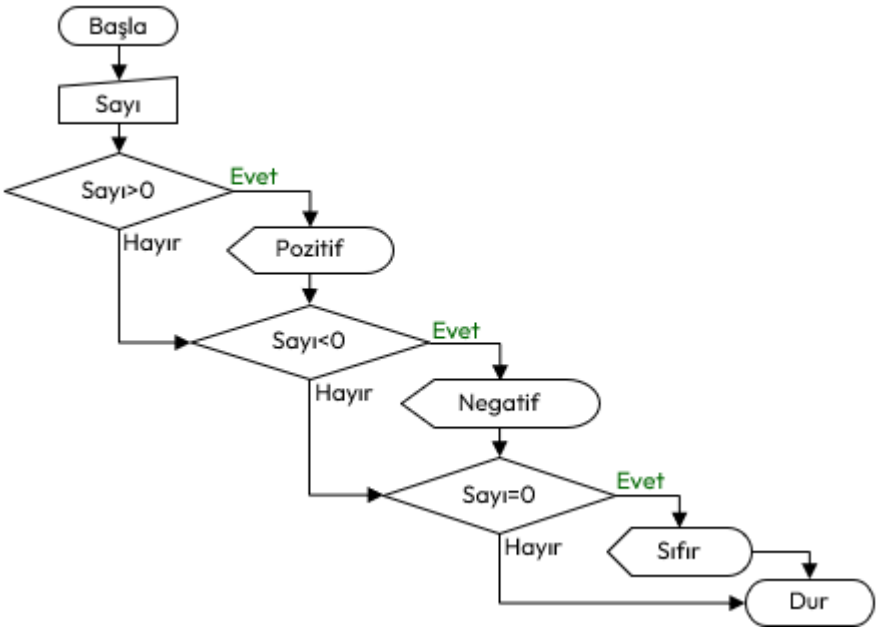
Bazen programcı tarafından aşağıdaki gibi if talimatları yazabilir.

```
if (1)
        printf("Bu metin konsola her zaman yazılır.");
if (0)
        printf("Bu metin konsola hiçbir zaman yazılmaz!");
```

Örnek olarak klavyeden girilen bir sayının işaretini bulan bir C programı yazalım;

```
#include <stdio.h>
int main() {
        int sayi;
        printf("Sayı Giriniz:");
        scanf("%d",&sayi);
        if (sayi>0) // sayının pozitif olup olmadığı test ediliyor.
                printf("Pozitif"); // Burada sayı>0 olduğu biliniyor.
        if (sayi<0) // sayının negatif olup olmadığı test ediliyor.
                printf("Negatif"); // Burada sayı<0 olduğu biliniyor.
        if (sayi==0) // sayının sıfır olup olmadığı test ediliyor.
                printf("Sıfır"); // Burada sayı=0 olduğu biliniyor.
        //Başka ihtimal olsaydı buraya gelecekti.
        return 0;
}
```

Bunun için oluşturacağımız akış diyagramı aşağıdaki şekilde olacaktır.



Şekil 6.4: Bir Sayının İşaretini Bulan Akış Diyagramı

§6.3.2 If-Else Talimatı

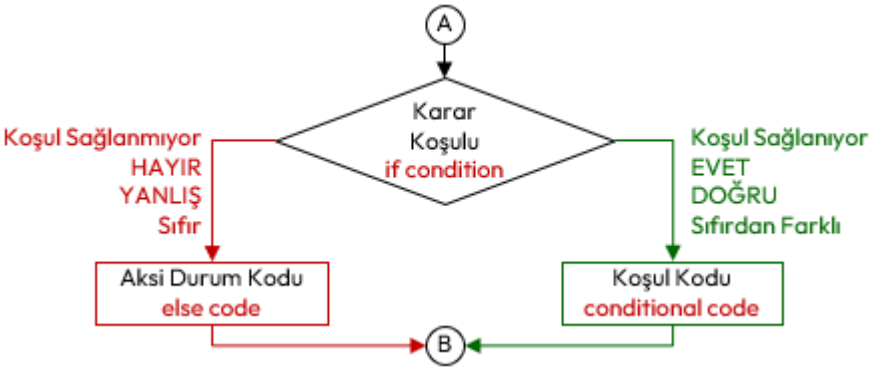
If-Else Talimatı, bir başka karar verme (decision making) talimatı olup bir koşula bağlı olarak programın icrasını değiştirir. Sözde kodu aşağıda verilmiştir;

```
if (karar-koşulu)
    koşul-kodu;
else
    aksi-durum-kodu;
```

If-Else talimatında, **karar koşulu** (condition) ifadesi DOĞRU ise **koşul kodu** (conditional code) icra edilir, karar koşulu YANLIŞ ise **aksi durum kodu** (else code) icra edilir.

Aşağıda verilen örnek programı inceleyelim;

```
1 #include <stdio.h>
2 int main() {
3         int yas=65;
4         char cinsiyet='K';
```



Şekil 6.5: If-Else Talimatı İcra Akışı

```
5         if (yas<50)
6                 printf("Genç");
7         else
8                 printf("Yaşlı");
9         return 0;
10 }
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

1. Birinci Satırda `printf` fonksiyonu için `stdio.h` başlığı koda dahil edilmiştir.
2. İkinci satırda programın başlayacağı ana fonksiyon tanımlanmıştır.
3. Üçüncü ve dördüncü satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
4. Beşinci satırda bir `if` talimatı bulunmaktadır. **Koşul kodu** (condition), `yas<50` ifadesiyle test edilecektir. Test sonucu 0 yani YANLIŞ olduğundan altıncı satırdaki koşul kodu (conditional code) talimatı **icra edilmez.** Bunun yerine `else` sonrası sekizinci satırdaki aksi durum kodu (else code) talimatı **icra edilir.** Yani `printf("Yaşlı");` talimatı ile konsola `"Yaşlı"` yazılır.
5. Son olarak dokuzuncu satırdaki `return 0;` ifadesi icra edilerek işletim sistemine dönülür.

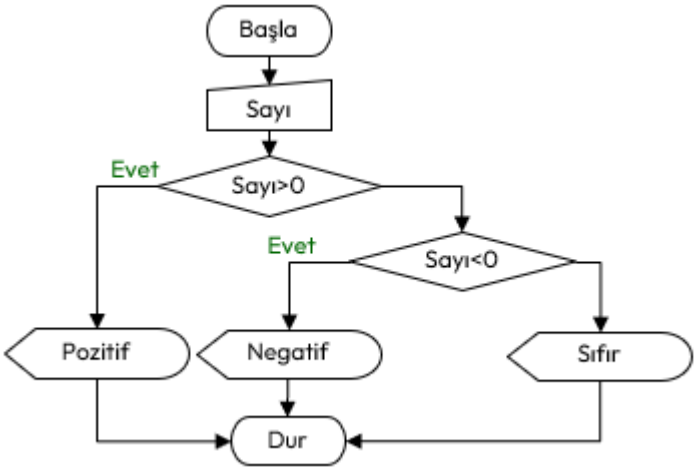
If talimatında olduğu gibi bu talimatta da **koşul kodu** (conditional code) ya da **aksi durum kodu** (else code) birden fazla talimattan oluşacak ise kod bloğu içine alınır. Buna ilişkin kod örneği aşağıda verilmiştir.

```
if (cinsiyet=='E') {
        if (yas<7)
        printf("Erkek Çocuk ");
        if (yas<18)
        printf("Genç Delikanlı ");
        if (yas>60)
        printf("Yaşlı Adam ");
} else {
        if (yas<3)
        printf("Kız Bebek ");
        if (yas<7)
        printf("Kız Çocuk ");
        if (yas<18)
        printf("Genç Kız ");
        if (yas>60)
        printf("Yaşlı Kadın ");
}
```

Bir sayının işaretini bulan C programının icra akışı olan Şekil 6.4 incelendiğinde ilk `if` talimatında yapılan test doğru çıkarsa geri kalan testlerin yapılmasına gerek kalmadığı anlaşılacaktır. Benzer şekilde ilk iki test geçilirse üçüncü if talimatındaki teste gerek yoktur. Bu durumu aşağıdaki akış diyagramında daha net görebiliriz;

Bu durumda aynı program `if-else` talimatlarıyla aşağıdaki şekilde yeniden yazabiliriz;

```
1 #include <stdio.h>
2 int main() {
3         int sayi;
4         printf("Sayi Giriniz:");
5         scanf("%d",&sayi);
6         if (sayi>0) // sayı için pozitif testi yapıldı
```



Şekil 6.6: Bir Sayının İşaretini Bulan If-Else Akış Diyagramı

```
7                printf("Pozitif"); // Burada sayı>0 olduğu biliniyor
8        else { // Burada sayı>0 olmadığı biliniyor
9                if (sayi<0) // sayı için negatif testi yapıldı
10                       printf("Negatif"); // Burada sayı<0 olduğu biliniyor
11               else // Bu noktada hem sayı>0 olmadığı ve hem de sayı <0 olmadığı biliniyor
12                       printf("Sıfır"); // Bu noktada sayı=0 olduğu biliniyor
13       }
14       return 0;
15 }
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- Eğer sayı pozitif girilirse;
  - Altıncı satırdaki if karar koşulunda `sayi>0` test edilip DOĞRU çıkacak ve sadece yedinci satırdaki koşul kodu olan `printf("Pozitif");` talimatı icra edilecektir. Sekizinci satırdan başlayan ve on üçüncü satırda biten aksi durum kodu icra edilmeyecektir.
  - Daha sonra on dördüncü satırdaki `return 0;` talimatı icra edilerek işletim sistemine dönülecektir.
- Eğer sayı negatif girilirse;
  - Altıncı satırdaki `if` karar koşulunda `sayi>0` test edilip YANLIŞ çıkacak ve sekizinci satırdan başlayan ve on üçüncü satırda biten aksi durum kodu icra edilecektir.
  - Bu blok içinde dokuzuncu satırdaki `if` karar koşulu `sayi<0` test edilip DOĞRU çıkacak ve onuncu satırdaki koşul kodu olan `printf("Negatif");` talimatı icra edilecektir. On ikinci satırlardaki aksi durum kodu icra edilmeyecektir.
  - Daha sonra on dördüncü satırdaki `return 0;` talimatı icra edilerek işletim sistemine dönülecektir.
- Eğer sayı sıfır girilirse;
  - Altıncı satırdaki `if` karar koşulunda `sayi>0` test edilip YANLIŞ çıkacak ve sekizinci satırdan başlayan ve on üçüncü satırda biten aksi durum kodu icra edilecektir.
  - Bu blok içinde dokuzuncu satırdaki `if` karar koşulu `sayi<0` test edilip YANLIŞ çıkacak ve onuncu satırdaki koşul kodu talimatı icra edilemeyecektir. Onun yerine on ikinci satırlardaki aksi durum kodu olan `printf("Sıfır");` icra edilecektir.
  - Daha sonra on dördüncü satırdaki `return 0;` talimatı icra edilerek işletim sistemine dönülecektir.

Ayrıca kod incelendiğinde her bir `if` ve `else` sonrasında tek talimat bulunmaktadır. Dolayısıyla blok kullanmaya gerek de yoktur. Bu durumda kodun nihai hali aşağıda verilmiştir.

```
#include <stdio.h>
int main() {
        int sayi;
        printf("Sayi Giriniz:");
        scanf("%d",&sayi);
        if (sayi>0)
                printf("Pozitif");
        else
                if (sayi<0)
                        printf("Negatif");
                else
```

```
                        printf("Sıfır");
        return 0;
}
```

§6.3.3 Sarkan Else

Aşağıdaki program örneği incelendiğinde `else`, hangi `if` talimatına aittir? Böyle bir kod programcı tarafından yazılabilir ve derleyici buna hiçbir sıkıntı çıkarmaz. Bu problem **sarkan else** (dangling else) problemi olarak bilinir.

```
if (cinsiyet=='E') if (yas<18) printf("Delikanlı"); else printf("Adam");
```

Bu durumda kodu aşağıdaki gibi okunaklı hale getirdiğimizde ilk `if` talimatının **koşul kodunun** (conditional code) ikinci `if-else` talimatı olduğu görülmektedir.

```
if (cinsiyet=='E')
        if (yas<18)
                printf("Delikanlı");
        else
                printf("Adam");
```

Kod bloğu kullanılmadan yazılan bu tip kodlarda `else` her zaman kendinden bir önceki `if` talimatına aittir.

> Sarkan else durumundaki mantıksal hataların önüne geçmek için
> **acemi kullanıcılar her zaman blok kullanmalıdır.**

```
 if (cinsiyet=='E') {
         if (yas<18) {
                 printf("Delikanlı");
         } else {
                 printf("Adam");
         }
}
```

§6.3.4 Switch Talimatı

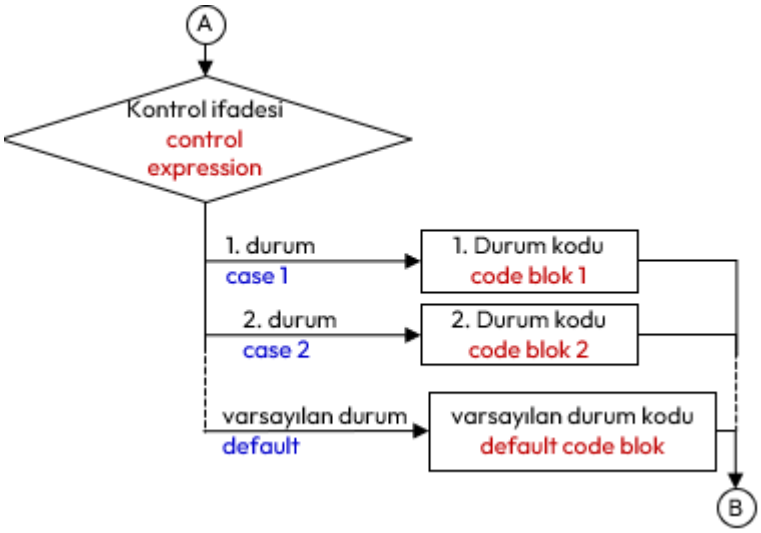
Switch talimatı, bir kontrol ifadesi (control expression) sonucunda birden fazla alternatif arasında seçim yapılmasını sağlar. Sözde kodu aşağıda verilmiştir;

```
switch (kontrolifadesi) { //switch bloğu başlangıcı
    case alternatif-1:
        alternatif-1-kodu;
        break; //break talimat zorunlu değil
    case alternatif-2:
        alternatif-2-kodu;
        break; //break talimat zorunlu değil
    // ...
    default:
        varsayılan-alternatif-kodu;
} //switch bloğu bitişi
```

> Switch talimatına ilişkin kurallar;
>
> 1. Kontrol ifadesi (control expression), **sonucu tamsayı olan ifadedir ve bir kez değerlendirilir**.
> 2. **Bloğu olmayan bir switch talimatı yazılamaz!**
> 3. **Her bir alternatif bir tamsayı değişmezi** (integer literal) izleyen ve iki nokta ile biten bir etiket (label) olarak tanımlanır.
> 4. **Alternatif hiçbir zaman değişken veya ifade (expression) olamaz!**
> 5. Blok için yazılan kod sonuna `break;` talimatı yazılarak `switch` bloğu dışına çıkılır. Zorunlu değildir, yazılmaz ise bir sonraki alternatif için yazılan kod icra edilir.
> 6. Tüm tam sayıları içerecek şekilde alternatiflerin tümü yazılmayabilir. Blok sonunda yer alacak şekilde üzerinde yer alan alternatifler dışında kalan tüm alternatifler için `default:` etiketli alternatif yazılabilir. Zorunlu değildir.

Aşağıda örnek olarak menü seçimi için yazılmış bir program örneği verilmiştir;

```
1 #include <stdio.h>
```



Şekil 6.7: Switch Talimatı İcra Akışı

```
2  int main() {
3          int menu;
4          printf("Menü İçin Rakam Giriniz:");
5          scanf("%d",&menu);
6          switch(menu) {
7                  case 1:
8                          printf("1 Numaralı Menü:\n");
9                          printf("Hamburger ve Ayran Hazırlanacak.\n");
10                         break;
11                 case 2:
12                         printf("2 Numaralı Menü:\n");
13                         printf("Patates ve Kola Hazırlanacak.\n");
14                         break;
15                 case 3:
16                 case 4:
17                         printf("3 veya 4 Numaralı Menü:\n");
18                         break;
19                 default:
20                         printf("1,2, 3 veya 4 Dışında Menü:\n");
21                         printf("Böyle bir Menümüz Yok!\n");
22         }
23         return 0;
24 }
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- Eğer `menu`, 1 girilirse;
  - 6.Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 1 olduğuna karar verilir.
  - Bu durumda 7.Satırdaki `case 1:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
    - İlk önce 8.Satırdaki `printf("1 Numaralı Menü:\n");` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 9.Satırdaki `printf("Hamburger ve Ayran Hazırlanacak.\n");` icra edilir ve bir sonraki talimata geçilir.
    - Ondan sonra 10.Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
  - Son olarak 23.Satırdaki `return 0;` talimatı icra edilerek işletim sistemine dönülür.
- Eğer `menu`, 2 girilirse;
  - 6.Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 2 olduğuna karar verilir.
  - Bu durumda 11.satırdaki `case 2:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
    - İlk önce 12.Satırdaki `printf("2 Numaralı Menü:\n");` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 13.Satırdaki `printf("Patates ve Kola Hazırlanacak.\n");` icra edilir ve bir

sonraki talimata geçilir.
    - Ondan sonra 14.Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
  - Son olarak 23.Satırdaki `return 0;` talimatı icra edilerek işletim sistemine dönülür.
- Eğer `menu`, 3 girilirse;
  - 6.Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 3 olduğuna karar verilir.
  - Bu durumda 15.Satırdaki `case 3:` etiketine gidilir ve bu alternatife ilişkin yazılmamıştır. Ancak ilk icra edilebilir koda kadar ilerlenir ve oradaki kodlar sırasıyla icra edilir;
    - İlk önce 17.Satırdaki `printf("3 veya 4 Numaralı Menü:\n");` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 18.satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
  - Son olarak 23.Satırdaki `return 0;` talimatı icra edilerek işletim sistemine dönülür.
- Eğer `menu`, 4 girilirse;
  - 6.Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 4 olduğuna karar verilir.
  - Bu durumda 16.Satırdaki `case 4:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
    - İlk önce 17.Satırdaki `printf("3 veya 4 Numaralı Menü:\n");` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 18.Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
  - Son olarak 23.Satırdaki `return 0;` talimatı icra edilerek işletim sistemine dönülür.
- Eğer `menu`,-1 , 0, 12 ve 300 gibi yukarıdaki alternatiflerden farklı girilirse;
  - 6.Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu düzenlenememiş bir durum olduğuna karar verilir.
  - Bu durumda 19.Satırdaki `default:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
    - İlk önce 20.Satırdaki `printf("1,2, 3 veya 4 Dışında Menü:\n");` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 21.Satırdaki `printf("Böyle bir Menümüz Yok!\n");` talimatı bizi `switch` icra edilir.
    - artık `switch` bloğunun sonuna ulaşıldığından bloktan çıkılır.
  - Son olarak 23.Satırdaki `return 0;` talimatı icra edilerek işletim sistemine dönülür.

Aşağıda klavyeden girilen bir sayının 4 rakamına bölünmesinde kalan üzerinden işlem yapan `switch` talimatı bir başka örnek program verilmiştir;

```c
#include <stdio.h>
int main() {
        int sayi;
        printf("Bir Sayı Giriniz:");
        scanf("%d",&sayi);
        switch(sayi%4) { //kontrol ifadesi bir işlem içerir
                case 3:
                        printf("Sayı 4 rakamına bölündüğünde kalan 3'tür.\n");
                        break;
                case 2:
                        printf("Sayı 4 rakamına bölündüğünde kalan 2'dir.\n");
                        break;
                case 1:
                        printf("Sayı 4 rakamına bölündüğünde kalan 1'dir.\n");
                        break;
                default: // Başka alternatif yoktur.
                        printf("Sayı 4 Rakamına TAM Bölünür.\n");
        }
        return 0;
}
```

§6.3.5 Üçlü Koşul İşleci

Daha önce İşleçler başlığı altında işlenenlere ek olarak, ardışık işlemlerde (sequential operation) if talimatlarına gerek kalmadan bir koşula göre ifadeler (expression) işlenecek ise **üçlü koşul işleci** (conditional tenary operator) kullanılır. Aşağıda üçlü işlecin iki sözdizimi verilmiştir;

```
koşul ? doğruifadesi : yanlışifadesi;
değişken = koşul ? doğruifadesi : yanlışifadesi;
```

Aşağıda oy kullanma durumu örneği bu işleçle işlenmiştir;

```
#include <stdio.h>
int main() {
        int yas;
        printf("Yaşınız?:");
        scanf("%d", &yas);
        (yas >= 18) ? printf("Oy Kullanabilirsin.") : printf("Oy Kullanamazsın!");
        return 0;
}
```

Aşağıda başka kullanımlarına ilişkin farklı kod örneği verilmiştir;

```
#include <stdio.h>
int main() {
        int sayi,tek,cift;
        printf("Bir Sayı Giriniz: ");
        scanf("%d", &sayi);
        tek= (sayi%2) ? 1 : 0;
        cift= tek ? 0 : 1;
        int sonuc1=tek ? sayi+30 : sayi+40;
        int sonuc2=cift ? sayi*30 : sayi*40;
        //...
        return 0;
}
```

6.4 İlişkisel Döngüler

İşlemciler iyi bir sayıcıdırlar. CPU içindeki kaydediciler (registers) sayma işlevini ve birçok matematiksel işlemi yapmak için de kullanılır.

Birçok problemlerin çözümüne, belli adımların tekrarlanması ile gidilir. Bu tip problemler şimdiye kadar olan yöntemlerle yapılırsa, tekrar tekrar aynı kodu yazmak zorunda kalırız. Konsola 10 defa ismimizi yazdıran bir program örneğini ele alalım.

```
#include <stdio.h>
int main() {
        printf("ILHAN\n");  //1
        printf("ILHAN\n");  //2
        printf("ILHAN\n");  //3
        printf("ILHAN\n");  //4
        printf("ILHAN\n");  //5
        printf("ILHAN\n");  //6
        printf("ILHAN\n");  //7
        printf("ILHAN\n");  //8
        printf("ILHAN\n");  //9
        printf("ILHAN\n");  //10
        return 0;
}
```

Program incelendiğinde aynı `printf` talimatının tekrar tekrar yazıldığını görürüz. **Bu istenmeyen bir durumdur.**

Yapısal programlamada ise yazdığımız bir kodu tekrar tekrar icra etmenin yolu **ilişkisel döngü** (relational loop) talimatlarını kullanırız.

§6.4.1 Sayaç Kontrollü Döngüler

Belirlenen bir sayaç değişkeninin istenilen bir değere ulaşıp ulaşmadığı kontrol ederek kodun tekrar tekrar icra edilmesi sağlanır. Yapısal programlama öncesinde tekrar edilecek kodun başına dönüş `goto` talimatıyla sağlanır. **Bu talimat icra sırasını etkileyen bir talimattır.**

Kodlama sırasında tekrar edilecek kodun başına iki nokta ile biten bir **etiket** (label) konulur. Etiketlere kimlik verilirken yine değişken kimliklendirme (identifier definition) kuralları uygulanır. Tekrar edilecek kodun sonunda ise **goto** talimatı etiketle birlikte kullanılır.

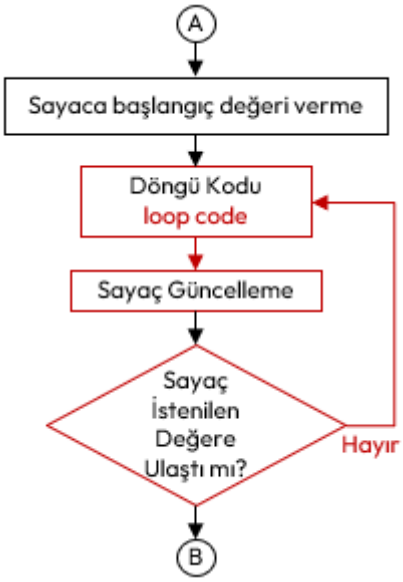
```
1 #include <stdio.h>
2 int main() {
3         int sayac=0;
```

```
4         Etiket: //Döngü Kodunun Başı ETİKETLENİR!
5         printf("ILHAN %d\n",sayac);
6         sayac++; //Sayaç Güncelleme
7         if (sayac<10) //Sayaç Kontrolü
8                 goto Etiket; // İstenilen değere ulaşılmamış ise Etiket'e git.
9         return 0;
10 }
```

Örneği şu şekilde açıklayabiliriz;

- 3.Satırda tekrar edilecek koda girmeden sayacımıza ile değer verdik.
- 4.Satıra geri dönülecek kodun başını işaretleyen bir etiket konulmuştur.
- 5.Satırda tekrarlanacak talimat yazılmıştır. Birden fazla talimat da yazabiliriz.
- 6.Satırda sayacımızı güncelledik.
- 7.Satırda sayacın istenilen değere ulaşıp ulaşmadığını kontrol ettik. Eğer ulaşmamış ise 8.Satırdaki `goto Etiket;` talimatıyla 4.Satıra döndük.

Sonuç olarak beşinci ile sekizinci satır arasında dört talimat içeren **mantıksal satır** (logical sequence) olmasına karşın 40 adet **fiziksel satır** (physical sequence) icra edilmiştir



Şekil 6.8: Sayaç Kontrollü Döngü İcra Akışı

C programlama dilinde ara seviye bir dil olduğundan `goto` talimatı desteklenir. 1968 Yılında Edsger W. Dijkstra `GOTO/JUMP TO` ifadelerini zararlı olarak ilan edilmiştir.

Yukarıda verilen döngü kodu örneğindeki gibi arapsaçı kodlamadaki `goto` talimatı yerine, **`while`, `do-while` ve `for` kontrol talimatları yapısal programlamaya eklenmiştir.**

§6.4.2 While Talimatı

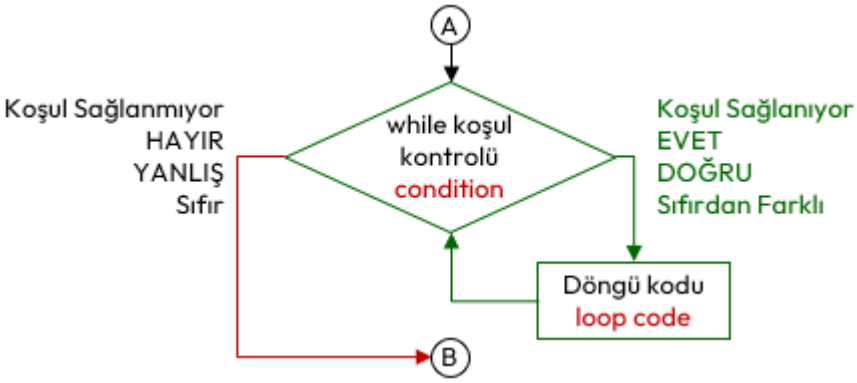
While döngüsü, koşul (condition) DOĞRU olduğu sürece döngü kodu (loop code) icra edilir. Tekrarlanacak kod, tek bir talimat (statement) olabileceği gibi bir kod bloğu da olabilir.

While döngüsünün sözde kodu aşağıda verilmiştir;

```
while (koşul)
    döngü-kodu;
```

Bu durumda 6.4.1 başlığında verilen döngü örneği, `while` talimatı ile aşağıdaki şekilde yazılabilir;

```
#include <stdio.h>
int main() {
        int sayac=0; //sayaca ilk değer verme
        while (sayac<10) { // döngü bloğu başlangıcı
                printf("ILHAN\n");
                sayac=sayac+1; //Sayaç Güncelleme
```



Şekil 6.9: While Talimatı İcra Akışı

```
        } //döngü bloğu bitişi
        return 0;
}
```

Sayaç güncelleme ifadesi koşul ifadesine eklenerek program daha da kısaltılabilir. Her koşul kontrolü sonrası sayaç artırılacaktır;

```
#include <stdio.h>
int main() {
        int sayac=0; //sayaca ilk değer verme
        while (sayac++<10) // hem sayaç güncelleme hem de koşul kontrolü
                printf("ILHAN\n"); // tek talimat olduğundan blok kullanılmadı
        return 0;
}
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu bir adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 10 adet fiziksel satır (physical sequence) icra edilmiştir.

§6.4.3 Do-While Talimatı

Do-While döngüsü, do ile while saklı kelimeleri arasındaki döngü kodu (loop code) koşul (condition) DOĞRU olduğu sürece tekrarlayarak icra eder. Tekrarlanacak kod tek bir talimat (statement) olabileceği gibi bir kod bloğu da olabilir. While döngüsünde koşul kontrolü en başta yapılır, bu döngüde ise döngü bloğu en az bir kez çalıştırıldıktan sonra koşul kontrolü yapılır.

Şekil 6.10: Do-While Talimatı İcra Akışı

Do-While talimatının sözde kodu aşağıda verilmiştir;

```
do   // "do" kelimesi goto talimatındaki etiket gibi davranır.
    döngü-kodu;
while (koşul);
```

Bu durumda 6.4.1 başlığında verilen döngü örneği,`do-while` talimatı ile aşağıdaki şekilde yazılabilir;

```
#include <stdio.h>
```

```
int main() {
        int sayac=0; //sayaca ilk değer verme
        do  {  // döngü bloğu
                printf("ILHAN\n");
                sayac=sayac+1; //Sayaç Güncelleme
        } while (sayac<10); // Koşul kontrolü
        return 0;
}
```

Sayaç güncelleme ifadesi koşul ifadesine eklenerek program aşağıdaki şekilde daha da kısaltılabilir.

```
#include <stdio.h>
int main() {
        int sayac=0; //sayaca ilk değer verme
        do
                printf("ILHAN\n"); // tek talimat olduğundan blok kullanıl
        while (sayac++<10); // hem sayaç güncelleme hem de koşul kontrolü
        return 0;
}
```

Yukarıdaki son örnek incelendiğinde ibir adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 10 adet fiziksel satır (physical sequence) icra edilmiştir.

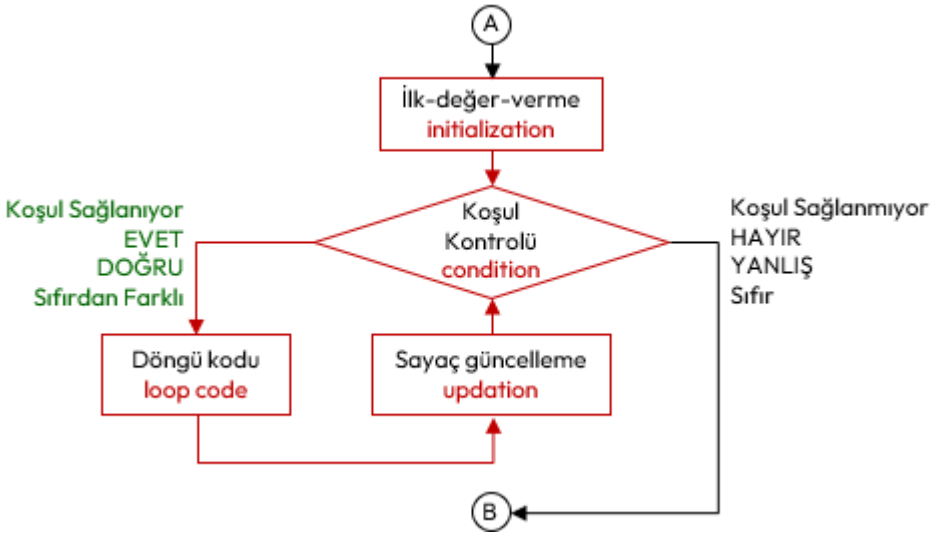
§6.4.4 For Talimatı

For döngüsü özellikle tekrar edilen işlemlerin sayısı belli olduğunda kullanılan bir döngü yapısıdır. Sayaç kontrollü döngüler için en iyi seçimdir. Sözde kodu aşağıda verilmiştir.

```
for (ilk-değer-verme; koşul-kontrolü; sayaç-güncelleme)
    döngü-kodu;
```

For talimatı aşağıdaki şekilde icra edilir;

1. **İlk değer verme ifadesi bir kez icra edilir.**
2. Sonra koşul kontrolü yapılır. Eğer **test sonucu DOĞRU ise döngü kodu icrasına geçilir.** Aksi halde döngüden çıkılır.
3. Döngü kodu icra edilir.
4. **Döngü kodu icrası bitince sayaç güncellemeleri yapılır ve tekrar ikinci adımdaki koşul kontrolüne dönülür.**



Şekil 6.11: For Talimatı İcra Akışı

Bu durumda 6.4.1 başlığında verilen döngü örneği, `for` talimatı ile aşağıdaki şekilde yazılabilir.

```
#include <stdio.h>
int main() {
        int sayac;
        for (sayac=1; sayac<10; sayac++)
                printf("ILHAN\n");
```

```
        return 0;
}
```

Buradaki döngü mantıksal olarak iki satır talimattan oluşmaktadır. Ancak fiziksel olarak 10 satır icra edilmiştir. Aşağıda konsola yazdığımız her bir satırın başına satır numarası koyan bir program gösterilmektedir.

```
#include <stdio.h>
int main() {
        int sayac;
        for (sayac=0; sayac<10; sayac++)
                printf("%02d-ILHAN\n", sayac+1);
        return 0;
}
/*Program Çıktısı:
01-ILHAN
02-ILHAN
03-ILHAN
04-ILHAN
05-ILHAN
06-ILHAN
07-ILHAN
08-ILHAN
09-ILHAN
10-ILHAN
*/
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu bir adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 10 adet fiziksel satır (physical sequence) icra edilmiştir.

For döngüsü bir sayaç ile çalışılabileceği gibi birden fazla sayaç ile de çalışılabilir. Hem ilk değer verme hem de sayaç güncellemede birden fazla sayaca ilişkin işlem yapılabilir. Bunun için **virgül işleci** (comma operator) kullanılır.

```
#include <stdio.h>
int main() {
        int sayac1,sayac2;
        for (sayac1=0,sayac2=10; sayac1<10; sayac1++,sayac2--)
                printf("%02d-%02d-ILHAN\n",sayac1,sayac2);
        return 0;
}
/*Program Çıktısı:
00-10-ILHAN
01-09-ILHAN
02-08-ILHAN
03-07-ILHAN
04-06-ILHAN
05-05-ILHAN
06-04-ILHAN
07-03-ILHAN
08-02-ILHAN
09-01-ILHAN
*/
```

Aşağıda klavyeden girilen iki sayı aralığında tek ve çift sayıların toplamını bulan ve ekrana yazan programı verilmiştir.

```
#include <stdio.h>
int main() {
        int sayac, sayi1,sayi2;
        int tekToplam=0,ciftToplam=0;
        printf("İki Sayı Giriniz (10-25) gibi: ");
        scanf("%d-%d",&sayi1,&sayi2);
        if (sayi2<sayi1) { //sayi2, sayi1 den büyük olmalı.
                int temp=sayi1; //küçük ise yer değiştiriyoruz.
                sayi1=sayi2;
```

```
                sayi2=temp;
        }
        for (sayac=sayi1; sayac<=sayi2; sayac=sayac+1) {
                if (sayac%2==1) //sayac tek mi?
                        tekToplam+=sayac;
                else
                        ciftToplam+=sayac;
        }
        printf("Tektoplam: %d\nCiftToplam: %d\n", tekToplam, ciftToplam);
        return 0;
}
/*Program Çıktısı:
İki Sayı Giriniz (10-25) gibi: 9-17
Tektoplam: 65
CiftToplam: 52
*/
```

Bir ve kendisinden başka tam böleni olmayan sayıya **asal sayı** denir. Klavyeden girilen pozitif bir tam sayının asal olup olmadığını ekrana yazdıran C programı aşağıdaki şekilde kodlanabilir;

```
#include <stdio.h>
int main()
{
        int sayi,asal=1;
        printf("Bir Sayı Giriniz:");
        scanf("%d",&sayi);
        for (int sayac=sayi-1; (sayac>1)&&(asal==1); sayac--)
                if (sayi%sayac==0)
                        asal=0;
        if(asal)
                printf("Girilen %d sayısı asaldır.",sayi);
        else
                printf("Girilen %d sayısı asal değildir.",sayi);
        return 0;
}
```

Klavyeden girilen pozitif bir tam sayının asal çarpanlarını ekrana yazdıran C programı aşağıdaki şekilde kodlanabilir;

```
#include <stdio.h>
int main()
{
        int sayi;
        printf("Bir Sayı Giriniz:");
        scanf("%d",&sayi);
        for (int sayac=sayi; sayac>0; sayac--)
                if (sayi%sayac==0)
                        printf("Asal çarpan:%d\n",sayac);
        return 0;
}
```

§6.4.5 İç İçe Döngü Kodu

Döngüler içinde yer alan döngü kodu bir başka döngüyü içerebilir. Örneğin ekrana satır ve sütun içeren bir şekil oluşturmak istediğimizde iç içe döngü (nested loop) kullanırız. Aşağıda buna ilişkin bir örnek verilmiştir. İlk **for** döngüsünün tekrarlanan kodu ikinci bir **for** döngüsüdür.

```
#include <stdio.h>
int main() {
        int satir,sutun;
        for (satir=0; satir<5; satir++) { //Dış Döngü Satırları Yazar
                for (sutun=0; sutun<4; sutun++) { //İç Döngü Sütunları Yazar
                        printf("*"); //İmleç, yazdırılan metnin sonunda olacaktır.
                }
                printf("\n"); //Satır işi bitince imleci yeni satıra geçir!
        }
```

```
        return 0;
}
/*Program Çıktısı:
****
****
****
****
****
*/
```

Bir başka örnek olarak elemanları, satır ve sütun çarpımları olan 4x5 boyutlarındaki matrisi ekrana yazdıran program aşağıda verilmiştir;

```
#include <stdio.h>
int main() {
        int satir,sutun;
        //Başlığı Yazdıran Kısım
        printf("   Sütun:\t");
        for(sutun=0; sutun<3;sutun++)
        printf ("%3d ",sutun+1);
        printf("\n");
        //Matrisi Yazdıran Kısım
        for (satir=0; satir<5; satir++) { //Satır için
                printf("%2d.Satir:\t",satir+1); //İmleç, yazdırılan metnin sonunda olacaktır.
                for(sutun=0; sutun<3;sutun++) //Sütun için
                        printf ("%03d ",sutun); //İmleç, yazdırılan metnin sonunda olacaktır.
                printf("\n"); //Satır işi bitince imleci yeni satıra geçir!
        }
        return 0;
}
/*Program Çıktısı:
Sütun:          1   2   3
1.Satir:        000 001 002
2.Satir:        000 001 002
3.Satir:        000 001 002
4.Satir:        000 001 002
5.Satir:        000 001 002
*/
```

§6.4.6 Gözcü Kontrollü Döngüler

Döngüler her zaman bir sayaca bağlı çalıştırılmaz. Bir döngüden çıkış koşulu döngü kodu içerisinde üretilen bir değere bağlı ise bu değere **gözcü değeri** (sentinel value) adı verilir. Buradaki gözcü değeri beklenen bir değer dışındaki bir değerdir. Buna örnek olarak klavyeden `0` girilene kadar girilen rakamları toplayan bir program verilmiştir.

```
#include <stdio.h>
int main()
{
        int sayi; /* okunan tamsayı */
        int toplam=0; /* girilen sayıların toplamı. başlangıçta 0 olmalı!*/
        do
        {
                printf("Bir sayı giriniz (Bitirmek için 0): ");
                scanf("%d",&sayi);
                toplam+=sayi; // sayi 0 girildiğinde toplam etkilenmez!
        } while (sayi != 0); //sentinel value 0
        printf("Girilen Sayıların Toplamı: %d",toplam);
        return 0;
}
/*Program Çıktısı:
Bir sayı giriniz (Bitirmek için 0): 10
Bir sayı giriniz (Bitirmek için 0): 20
Bir sayı giriniz (Bitirmek için 0): 30
Bir sayı giriniz (Bitirmek için 0): 0
```

```
Girilen Sayıların Toplamı: 60
*/
```

Aşağıda pozitif sayı girilmesini sağlayan bir başka kod örneği verilmiştir.

```
#include <stdio.h>
int main() {
        int sayi;
        do /* Pozitif girilmesini zorluyoruz */
        {
                printf("Lütfen pozitif sayı giriniz: ");
                scanf("%d",&sayi);
                if (sayi<=0)
                        printf("HATA: Pozitif Sayı GİRMEDİNİZ!\n");
        } while (sayi <= 0);
        printf("Girilen pozitif tamsayı: %d \n",sayi);
        return 0;
}
/*Program Çıktısı:
Lütfen pozitif sayı giriniz: -1
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: 10
Girilen pozitif tamsayı: 10
*/
```

Aynı örnek bir başka şekilde aşağıdaki gibi kodlanabilir. **Ancak burada beşinci ve altıncı satırlar ile onuncu ve on birinci satırların aynı olduğuna ve kod tekrarı yapıldığına dikkat ediniz!**

```
1  #include <stdio.h>
2  int main() {
3          int sayi;
4          printf("Lütfen pozitif sayı giriniz: ");
5          scanf("%d",&sayi);
6          while (sayi <= 0) /* Pozitif girilmesini sağlıyoruz */
7          {
8                  printf("HATA: Pozitif Sayı GİRMEDİNİZ!\n");
9                  printf("Lütfen pozitif sayı giriniz: ");
10                 scanf("%d",&sayi);
11         }
12         printf("Girilen pozitif tamsayı: %d \n",sayi);
13         return 0;
14 }
```

6.5 Dallanmalar

Dallanmalar (jump), bir döngünün yada fonksiyonun icrasını kesen ve bir başka talimata yönlendiren talimatlardır.

§6.5.1 Continue ve Break Talimatları

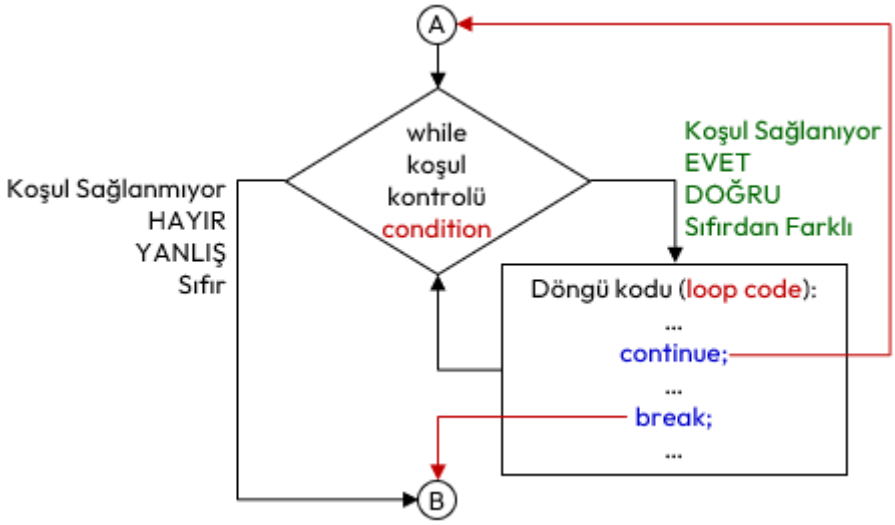
Continue talimatı , yalnızca geçerli yinelemeyi (iteration) sonlandırır ve sonraki yinelemelerle devam eder. Yani bu talimat, bir sonraki yinelemeyi yapmak için kullanılır. Dolayısıyla sadece `while`, `do-while` ve `for` döngü kodları (loop code) içinde kullanılabilir.

Break talimatı , döngüyü sonlandırır ve program icrası döngü sonrası ilk talimattan devam eder. Bunu daha önce `switch` talimatında görmüştük. Bu talimat, aynı şekilde `while`, `do-while` ve `for` döngüleri ile kullanılabilir.

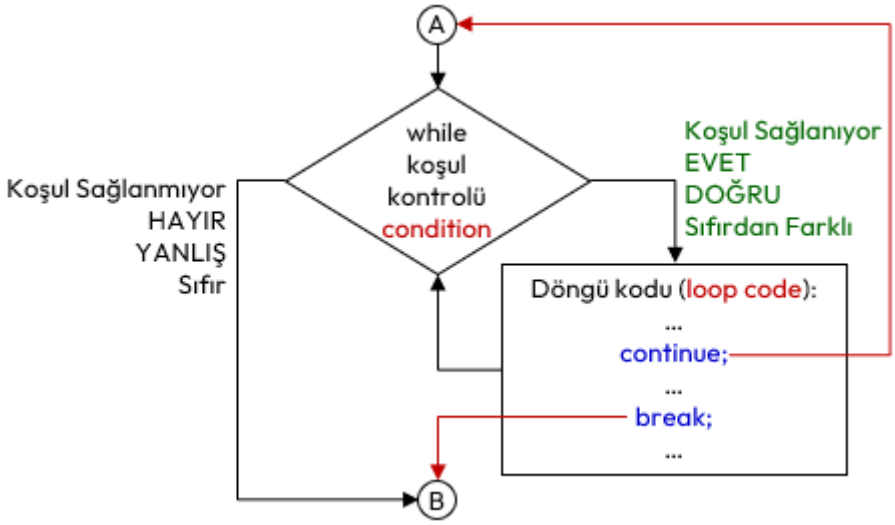
Bu talimatlarının döngü kodlarında kullanılması halinde icra akışı Şekil 6.12'de verilmiştir.

Bu talimatlar döngü kodu içinde amacımıza uygun olarak birden çok kullanabiliriz. Aşağıda 0 ile 15 arasındaki sayıların beşe bölünenleri ve 10 dışındaki sayılar ekrana yazdıran bir program örneği verilmiştir.

```
#include <stdio.h>
int main() {
        int i;
        for (i=0; i<=15; i=i+1) {
                if (i<7) //i'nin değeri 7 den küçük ise
```



(a) While



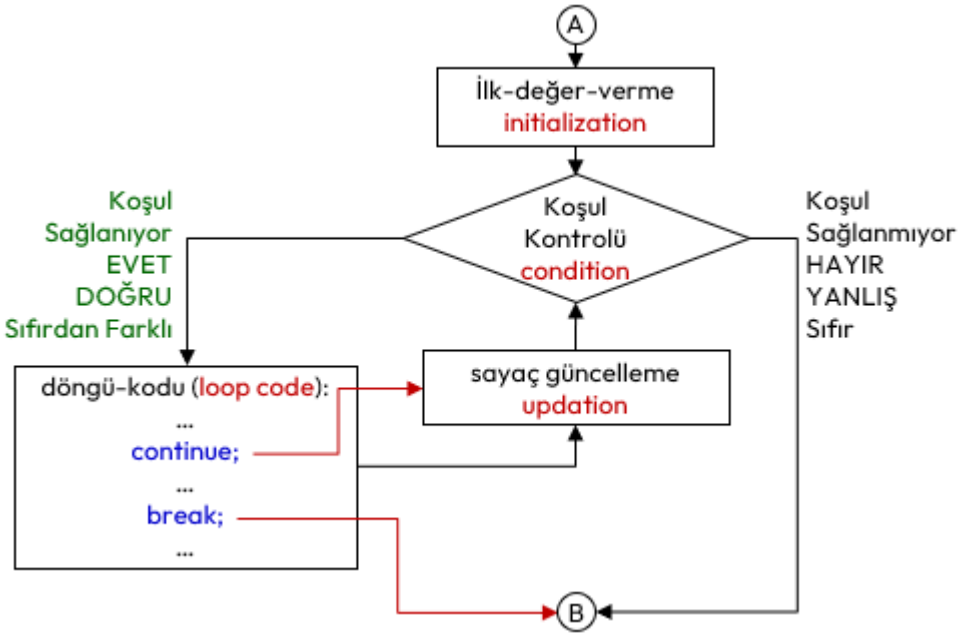
(b) Do-While

Şekil 6.12: While ve Do-While Döngülerinde Break ve Continue Talimatları İcra Akışı

```
                continue;  //bir sonraki yinelemeye (iteration) geç
        if (i==10) //i'nin değeri 10 ise
                continue;  //bir sonraki yinelemeye (iteration) geç
        if (i%5==0) // i'nin değeri 5 in katı ise
                continue;  //bir sonraki yinelemeye (iteration) geç
        printf ("i = %d \n",i);
    }
    return 0;
}
/*Program Çıktısı:
i = 7
i = 8
i = 9
i = 11
i = 12
i = 13
i = 14
*/
```

Yukarıdaki örnek incelendiğinde döngü bloğu üç adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 42 adet fiziksel satır (physical sequence) icra edilmiştir. Bazı `continue` talimatları `printf` talimatının icrasını engellemiştir.

Aşağıda pozitif sayı girilmesini zorlayan **sonsuz döngü** (infinitive loop) içeren program verilmiştir. Prog-



Şekil 6.13: For Döngüsünde Break ve Continue Talimatları İcra Akışı

ramda döngü, pozitif sayı girildiğinde **break;** talimatı ile kırılmaktadır.

```
#include <stdio.h>
int main() {
        int sayi;
        int pozitifSayi=0;
        while (1) { /* Sonsuz döngü */
                printf("Lütfen pozitif sayı giriniz: ");
                scanf("%d",&sayi);
                if (sayi<=0)
                        printf("HATA: Pozitif Sayı GİRMEDİNİZ!\n");
                else
                        break; //döngüden çık
        }
        pozitifSayi=sayi;
        printf("Girilen pozitif tamsayı: %d \n",pozitifSayi);
        return 0;
}
```

Bu talimatları dikkatli kullanmalıyız. Bazı durumlarda sonsuz döngüye girme (infinitive loop) durumu tüm döngü talimatlarında yaşanabilir. `for` Döngüsünde bu durum farklıdır. `continue;` talimatı sonrasında bir sonraki yineleme için önce sayaç güncelleme ifadeleri icra edilir ve sonra koşul testi yapılır.

§6.5.2 Return Talimatı

Yapısal programlamada ana fonksiyonun (main function) sonunda çokça kullandığımız bu talimat, bir fonksiyonun üreteceği ya da belirlenen değeri geri döndürür. Kullanıldığı yerden fonksiyon bloğu dışına çıkılır. Aşağıda üç defa negatif sayı girilmesi halinde programı sonlandıran bir program verilmiştir.

```
#include <stdio.h>
int main() {
        int sayi;
        int pozitifSayi=0;
        int sayac=0; //kaç defa pozitif olmayan sayı girildi.
        do // Pozitif girilmesini zorluyoruz
        {
                printf("Lütfen pozitif sayı giriniz: ");
                scanf("%d",&sayi);
                if (sayi<=0) {
                        printf("HATA: Pozitif Sayı GİRMEDİNİZ!\n");
                        sayac++;
                        if (sayac==3) {
```

```
                                printf("Üç defa negatif sayı girdiniz. ");
                                printf("Programdan Çıkılıyor.\n");
                                return 1; // ana fonksiyondan çıkılarak işletim sistemine 1
                                 ↪  gönderiliyor.
                        }
                }
        } while (sayi <= 0);
        pozitifSayi=sayi;
        printf("Girilen pozitif tamsayı: %d \n",pozitifSayi);
        return 0;
}
/*Program Çıktısı:
Lütfen pozitif sayı giriniz: -1
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -2
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -3
HATA: Pozitif Sayı GİRMEDİNİZ!
Üç defa negatif sayı girdiniz. Programdan Çıkılıyor.

...Program finished with exit code 1
*/
```

§6.5.3 Goto Talimatı

Tanımlı bir etikete program akışını yönlendiren `goto` talimatıdır. 6.4.1 başlığında örneği verilmiştir.

6.6 Düşün ve Çöz

- Düşün: Sarkan else (dangling else) problemi nedir? Bu problemi önlemek için neden her zaman süslü parantez kullanılması önerilir?
- Düşün: while ve do-while döngüleri arasındaki farkı, bir kullanıcıdan şifre isteme senaryosu üzerinden karşılaştırınız.
- Düşün: **switch-case** yapısında **break;** kullanılmadığında oluşan başarısız olma (fall-through) durumunu bir hata değil, bir özellik olarak nasıl kullanabiliriz?
- Çöz: Kullanıcıdan sürekli sayı alan ve `0` girildiğinde o ana kadar girilen en büyük sayıyı ve sayıların ortalamasını ekrana basan bir gözcü kontrollü döngü (sentinel value) tasarımı yapınız.
- Çöz: İç içe döngüler kullanarak ekrana 1'den 10'a kadar olan sayıların çarpım tablosunu düzgün bir formatta yazdırınız.
- Çöz: Kullanıcı negatif bir sayı girene kadar sayı alan ve girilen tek sayıların toplamını, çift sayıların ise adedini ekrana basan programı yazınız.